CostGuard: Policy-Governed Cost Anomaly Intelligence for Cloud DevOps Pipelines

Tejas Sudhakar Kamble

April 2026

Abstract

Cloud DevOps teams optimize delivery speed aggressively, but actionable cost signals often appear only after release decisions are already executed. This paper presents CostGuard PADE, a policy-governed anomaly intelligence system that moves FinOps control from retrospective reporting into CI/CD runtime. CostGuard combines two model views: an attention-enhanced residual BiLSTM branch for stage-sequence behavior and a GATv2 branch for dependency-aware graph behavior. Their outputs are integrated through adaptive ensembling, where weighted fusion is applied only when branch calibration is favorable and `lstm_only` routing is selected otherwise. The resulting risk signal is mapped to Cost Risk Score (CRS) and evaluated by an Open Policy Agent (OPA) governance layer that emits auditable actions (`ALLOW`, `WARN`, `AUTO_OPTIMISE`, `BLOCK`) with deterministic inline fallback.

Evaluation follows a fixed ten-seed protocol (42–132) across lifelong domains D0 (Synthetic), L1 (TravisTorrent), and L2 (BitBrains). Mean ensemble F1/PR-AUC is 0.8962/0.9648 (D0), 0.9087/0.9419 (L1), and 0.9606/0.9763 (L2), with low variance in real domains and full 10/10 trial completion. Runs were executed under an RTX 3050 4-GB profile using memory-safe controls: 30 epochs, patience 5, adaptive batch downshift, bounded preprocessing, and phase-level VRAM flush checkpoints. The core outcome is practical rather than only architectural: anomaly models become operationally meaningful when coupled with policy automation, traceable decisions, and reproducible evidence.

Keywords: FinOps, CI/CD, anomaly detection, LSTM, GATv2, Open Policy Agent, policy-as-code, reproducibility.

This manuscript is derived from the author's MSc research work on CostGuard PADE. The technical contributions are presented here in a consolidated single-author research format, without academic submission front matter.

1 Introduction

Cloud-native CI/CD pipelines now execute a large portion of software lifecycle cost: build runners, integration tests, security scans, image creation, and staged deployment. Teams have strong quality and velocity telemetry, but financially actionable feedback is typically delayed until billing reconciliation. This mismatch creates a structural FinOps problem: the cost signal arrives after the decision window closes.

CostGuard targets this gap by embedding cost anomaly intelligence directly into the delivery control loop. Instead of producing only post hoc dashboards, it couples anomaly scoring to policy decisions that can warn, optimize, or block risky stage execution. The goal is not only predictive performance; the goal is accountable operational behavior.

The paper is guided by four research questions:

- **RQ1:** Can a temporal-graph hybrid model maintain reliable anomaly quality across heterogeneous domains?
- **RQ2:** Can policy-as-code convert model scores into auditable runtime actions for CI/CD governance?
- **RQ3:** Can strict seed-wise reproducibility be preserved under commodity 4-GB GPU constraints?
- **RQ4:** Can the resulting approach be delivered as a practical engineering system, not only an offline benchmark?

Contributions are fourfold: (i) a dual-view anomaly model (LSTM + GATv2) with adaptive fusion, (ii) CRS-to-policy actioning through OPA with deterministic fallback, (iii) a fixed ten-seed D0→L1→L2 protocol with immutable aggregates, and (iv) a full implementation path using FastAPI, PostgreSQL, and Streamlit for operational adoption.

2 Related Work

2.1 Sequence and Graph Foundations

LSTM remains a standard approach for long-range dependency learning [1]. That property is important for cost-risk modeling because economically significant effects often emerge several stages after a triggering event. Graph Attention Networks learn edge influence rather than assuming uniform neighborhood impact [2], which is suitable for CI/CD DAGs where transitions are not equally consequential.

Related graph work also shows value when dependency structure is incomplete or weakly labeled [3, 4]. These insights support using graph reasoning in pipeline telemetry where link quality may vary by project, stage metadata, or instrumentation maturity.

2.2 FinOps and Cost Optimization

FinOps studies emphasize observability, optimization governance, and standardized usage semantics [10, 11, 15, 17]. They provide strong motivation for action-oriented cost control, but many reports stop at architecture patterns or case narratives. Reproducible, seed-controlled ML evidence linked to deployment decisions remains limited.

2.3 CI/CD Governance and Operational Control

ML-assisted CI/CD studies report improvements in software delivery efficiency [5, 6, 7, 8, 9]. Governance-oriented work highlights rule-bounded automation and policy simulation in cloud systems [14, 12, 13]. The unresolved gap is integration depth: model quality, policy decisions, and deployment-grade evidence are often evaluated in isolation.

Table 1: Positioning of prior literature against integrated CI/CD FinOps requirements.

Theme	Representative direction	Integration gap addressed in CostGuard
Temporal modeling	LSTM memory-based sequence learning [1]	Strong delayed-dependency learning, but no policy actioning path or runtime governance records.
Dependency modeling	Graph attention and latent link inference [2, 3, 4]	Strong relational reasoning, but limited seed-controlled CI/CD cost-risk evidence.
FinOps strategy	Cost optimization and service-oriented governance [10, 11, 15]	Mature governance concepts, but limited coupling to model-controlled release decisions.
CI/CD optimization	ML-based delivery acceleration studies [5, 6, 7, 8, 9]	Better throughput and quality, but weak policy provenance and auditable cost-risk controls.
Policy control	Agentic and elasticity policy frameworks [14, 12, 13]	Strong policy framing, but insufficient cross-domain anomaly benchmarking with frozen artifacts.

3 Methodology

3.1 Problem Framing

CostGuard models stage-level cost anomaly probability and maps it to governance actions. The methodological objective is to preserve predictive quality and reproducibility while maintaining deployment-safe controls in constrained hardware settings.

3.2 Temporal Branch (LSTM)

The sequence branch uses residual BiLSTM layers with attention. Given telemetry sequence x_t, LSTM gates are:

$$\begin{aligned} i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i), \\ f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f), \end{aligned} \tag{1}$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c), \quad c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t. \tag{2}$$

This branch captures delayed cost effects across ordered CI/CD stages.

3.3 Graph Branch (GATv2)

The graph branch models inter-stage dependency influence:

$$\alpha_{ij} = \mathrm{softmax}_j\left(a^\top \mathrm{LeakyReLU}(W[h_i \| h_j])\right), \tag{3}$$

$$h'_i = \sum_{j \in \mathcal{N}(i)} \alpha_{ij} W h_j. \tag{4}$$

Attention coefficients α_{ij} allow non-uniform transfer of risk signals along pipeline edges.

3.4 Adaptive Ensemble Logic

Validation PR-AUC calibrates branch weights:

$$\hat{p}_{ens} = w_L \hat{p}_{LSTM} + w_G \hat{p}_{GAT}, \tag{5}$$

$$w_L = \frac{\text{PR-AUC}_{LSTM}}{\text{PR-AUC}_{LSTM} + \text{PR-AUC}_{GAT}}, \quad w_G = 1 - w_L. \tag{6}$$

If branch alignment/calibration is unfavorable, routing switches to `lstm_only`. This mechanism protects quality when graph signal is weaker in specific domains.

3.5 CRS, Thresholds, and Policy Interface

CRS is the governance-facing risk scalar. In runtime, the backend computes model risk and evaluates it with context through policy checks. Default threshold actions are:

`ALLOW` → `WARN` → `AUTO_OPTIMISE` → `BLOCK`

at CRS cutoffs 0.50, 0.75, and 0.90.

The policy payload includes metrics (CRS, billed cost, latency), context (stage, branch, pull-request flags), and configurable policy bundle fields. OPA-first evaluation is paired with inline parity fallback to preserve continuity and provenance (`policy_source`).

3.6 Leakage and Reproducibility Controls

Three controls are treated as hard constraints:

- Train-only scaler fitting for each seed/domain split.
- Validation-only threshold optimization, frozen before test inference.
- Seed-isolated artifacts and immutable aggregate generation.

Together, these controls reduce calibration drift and cross-seed contamination risk.

4 System Design and Implementation

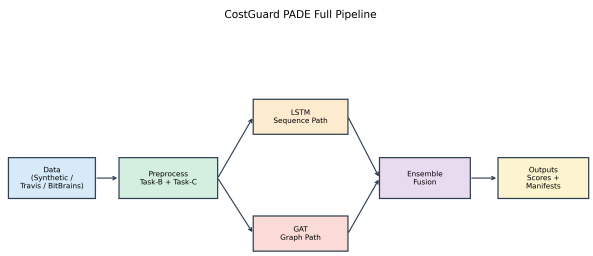

Figure 1: End-to-end CostGuard PADE workflow from telemetry ingestion to policy-governed decision output.

CostGuard is implemented as a deployable system, not a benchmark-only script:

- **FastAPI backend** for inference, governance, post-run import, and analytics APIs.
- **PostgreSQL** for seed metrics, policy outcomes, run manifests, and audit history.
- **OPA governance** for policy-as-code evaluation with inline fallback safety.
- **Streamlit dashboard** for operator visibility and governance control surfaces.

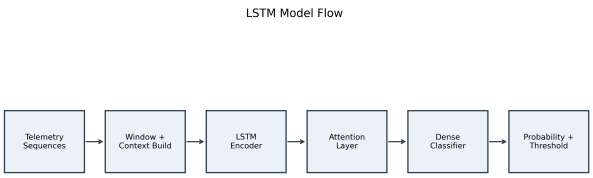

Figure 2: LSTM dataflow used in CostGuard sequence modeling.

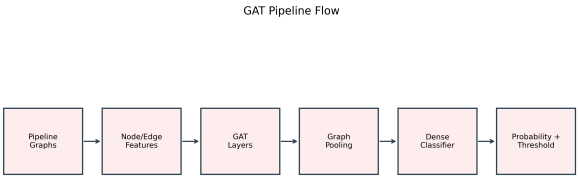

Figure 3: GATv2 dataflow used in dependency-aware pipeline modeling.

4.1 Governance Rule Behavior

Table 2: Policy bundle behavior in runtime governance.

Rule type	Runtime effect
Threshold gates	Escalation at CRS $\geq$ 0.50, 0.75, and 0.90
Context gates	Branch/stage-specific blocking (for example, production deploy from PR context)
Cost ceilings	Stage-specific cost overruns trigger `WARN` or `AUTO_OPTIMISE`
OPA fallback parity	Inline deterministic policy when OPA endpoint is unavailable

4.2 Evidence Persistence

Table 3: Representative persistence surfaces used for traceability.

Table/Artifact family	Purpose
`ieee_seed_runs`	Seed completion and execution metadata
`ieee_aggregate_metrics`	Domain-level summary metrics for reporting
`ieee_seed_domain_metrics`	Per-seed, per-domain model performance
`inference_events`	CRS, decision class, policy source, and reasons
`retraining_queue`	Queue-based continual readiness without online weight mutation

4.3 Engineering Decisions for Stable Execution

The training engine uses hardware-aware safety controls: adaptive batch ladders (`safe_batch`), bounded preprocess chunking, conservative worker settings on Windows, and VRAM flush between model phases. In source logs, this appears as repeated `[GUARDIAN-PROFILE]` and `[VRAM-FLUSH]` events. These controls were required to preserve full trial completion under constrained VRAM rather than to maximize isolated benchmark throughput.

5 Experimental Setup

5.1 Protocol and Domain Order

All experiments follow immutable lifelong order D0 Synthetic $\rightarrow$ L1 TravisTorrent $\rightarrow$ L2 BitBrains. Seed values are fixed to 42–132 at step 10, and aggregate outputs are computed from frozen per-seed artifacts.

Table 4: Dataset partitions used by sequence (Task-B) and graph (Task-C) training paths.

Domain	Task-B (LSTM)			Task-C (GATv2)		
	Train	Val	Test	Train	Val	Test
D0 Synthetic	43,750	9,375	9,375	43,750	9,375	9,375
L1 TravisTorrent	60,279	12,917	12,918	42,199	9,043	9,043
L2 BitBrains	174,365	37,364	37,364	34,582	7,410	7,411

5.2 Hardware and Runtime Envelope

Table 5: Runtime profile used across all reported seeds.

Setting	Value
GPU profile	NVIDIA RTX 3050 Laptop, 4095 MB VRAM
Learning rate	5×10^{-4}
Epoch horizon	30
Early stopping	Patience = 5
Seed protocol	10 fixed seeds (42–132)
Memory safety	Adaptive batching, chunked preprocessing, mixed-precision guard, VRAM flush

5.3 Why 30 Epochs and Patience 5

The epoch budget was selected after observing diminishing validation gains beyond 30 epochs under the same memory guard profile. Patience 5 balanced two competing concerns: avoiding premature stop during noisy validation transitions while preventing unnecessary compute after convergence.

5.4 Completion Timeline

Table 6: Per-seed completion evidence from frozen summary artifacts (UTC).

Seed	Started	Completed	Status
42	2026-04-22T06:33:37Z	2026-04-22T06:51:27Z	complete
52	2026-04-22T21:14:29Z	2026-04-22T23:56:46Z	complete
62	2026-04-23T11:35:00Z	2026-04-23T12:22:42Z	complete
72	2026-04-23T13:18:48Z	2026-04-23T16:43:04Z	complete
82	2026-04-23T19:26:09Z	2026-04-23T21:28:13Z	complete
92	2026-04-23T21:28:45Z	2026-04-24T00:27:41Z	complete
102	2026-04-24T00:28:14Z	2026-04-24T03:31:36Z	complete
112	2026-04-24T03:32:07Z	2026-04-24T06:28:49Z	complete
122	2026-04-24T06:29:21Z	2026-04-24T09:47:16Z	complete
132	2026-04-24T09:47:48Z	2026-04-24T12:57:43Z	complete

6 Results and Discussion

6.1 Aggregate Performance

Table 7: Aggregate metrics by domain and branch (mean over 10 seeds).

Domain	Model	F1@opt	ROC-AUC	PR-AUC	Precision	Recall	Threshold
Synthetic (D0)	LSTM	0.9128	0.9526	0.9766	0.9326	0.8941	0.2940
Synthetic (D0)	GAT	0.7468	0.7646	0.8209	0.7104	0.7932	0.3790
Synthetic (D0)	ENS	0.8962	0.9354	0.9648	0.9139	0.8799	0.3900
TravisTorrent (L1)	LSTM	0.9087	0.8133	0.9419	0.8454	0.9825	0.3710
TravisTorrent (L1)	GAT	0.7683	0.6939	0.8274	0.6246	0.9981	0.3370
TravisTorrent (L1)	ENS	0.9087	0.8133	0.9419	0.8454	0.9825	0.3710
BitBrains (L2)	LSTM	0.9606	0.9628	0.9763	0.9271	0.9966	0.1800
BitBrains (L2)	GAT	0.8760	0.5905	0.8281	0.7800	0.9990	0.2480
BitBrains (L2)	ENS	0.9606	0.9628	0.9763	0.9271	0.9966	0.1800

BitBrains delivers the strongest aggregate separability, while TravisTorrent maintains high-recall behavior desirable for anomaly containment workflows. Synthetic has higher variance than real domains, but still remains within bounded spread under the fixed protocol.

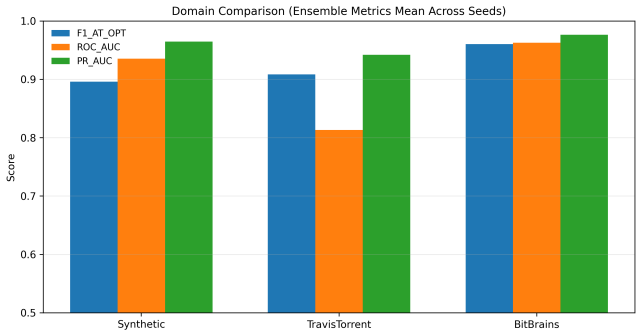

Figure 4: Domain-level metric comparison for LSTM, GATv2, and ensemble outputs.

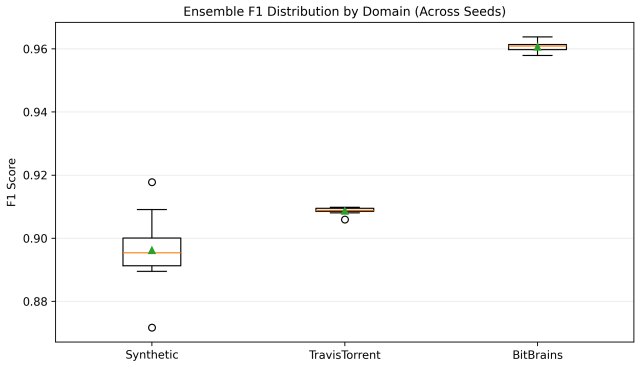

Figure 5: Distribution of ensemble F1 by domain.

6.2 Adaptive Strategy Behavior

Table 8: Stability and strategy routing statistics.

Domain	F1 std	Min–Max	Strategy usage
D0 Synthetic	0.0122	0.8717–0.9178	9 weighted / 1 lstm
L1 TravisTorrent	0.0011	0.9059–0.9098	10 lstm
L2 BitBrains	0.0017	0.9579–0.9637	10 lstm

The key practical finding is not that weighted fusion always dominates. The key finding is that adaptive routing avoids introducing branch-specific degradation. In D0, weighted fusion is beneficial for most seeds. In L1/L2, `lstm_only` preserves stronger calibrated behavior.

Table 9: Domain-level delta of ensemble F1.

Domain	ENS−LSTM	ENS−GAT	ENS threshold
D0 Synthetic	−0.0166	+0.1494	0.390
L1 TravisTorrent	0.0000	+0.1404	0.371
L2 BitBrains	0.0000	+0.0846	0.180

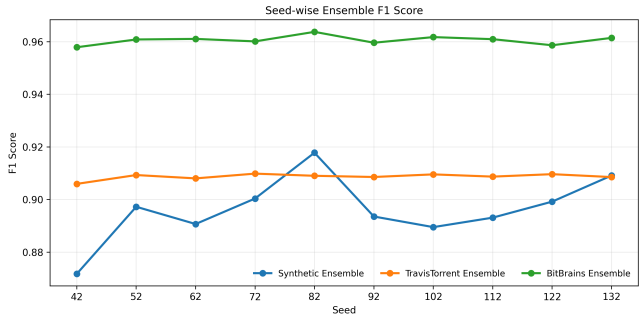

Figure 6: Seed-wise ensemble F1 across the canonical ten-seed set.

6.3 Curve-Level Analysis

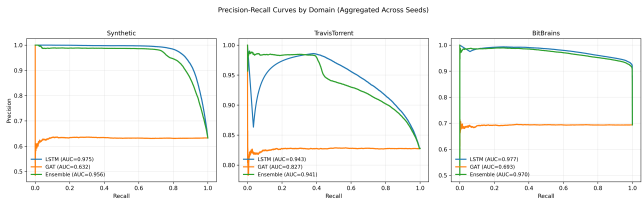

Figure 7: Precision-recall curves by domain.

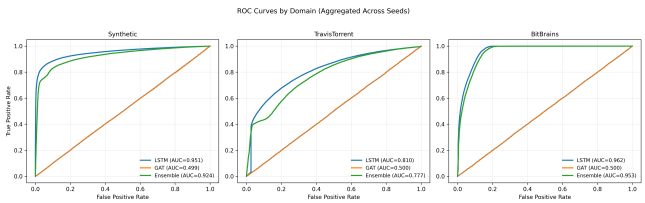

Figure 8: ROC curves by domain.

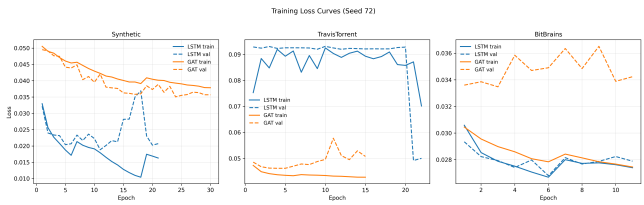

Figure 9: Representative training loss trajectory (seed 72).

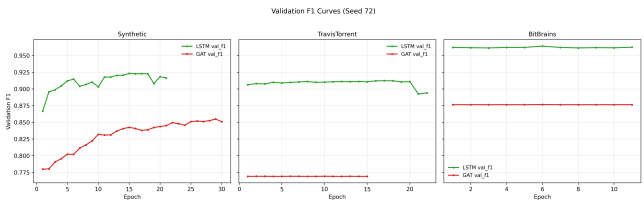

Figure 10: Representative validation F1 trajectory (seed 72).

PR profiles remain strong in high-recall regions, which is essential for anomaly screening under class imbalance. Training curves support the chosen epoch/patience design: after early convergence, additional epochs add cost without proportional quality gain.

6.4 Operational Metrics Beyond F1

Table 10: Average anomaly summary from imported test predictions.

Domain	Mean anomaly rate	Mean anomaly count	Mean test size
Synthetic	0.6273	5881.1	9375
TravisTorrent	0.6110	5525.0	9043
BitBrains	0.7487	5548.6	7411

6.5 Lifelong Retention

Backward Transfer remained 0.0 ± 0.0 across all ten seeds, indicating no measurable forgetting signal in the reported three-stage progression.

7 Practical Implications

1) Cost control in the decision window. CostGuard enables intervention before spend propagation. Teams can act during pipeline execution rather than after monthly cloud billing analysis.

2) CI/CD anomaly detection with governance context. Risk scoring is joined with stage and branch context. This reduces policy-blind actions where a low score might still be unsafe for sensitive deployment paths.

3) Deployment blocking as an explicit engineering control. `BLOCK` decisions are produced from clear threshold and rule matches, allowing platform teams to encode release safety and cost constraints without hidden manual logic.

4) OPA-driven policy automation. Policy bundles are externalized from model code, enabling governance updates (thresholds, stage ceilings, branch protections) independently from retraining cycles.

5) Evidence reuse across research and operations. The same artifacts that support paper reproducibility also support audit, incident review, and optimization planning. This dual-use evidence model improves maintainability.

6) Incremental rollout path for DevOps teams. Adoption can proceed in phases: monitor-only, then warn, then auto-optimize/block once governance confidence and incident response workflows are mature.

8 Limitations and Threats to Validity

Dataset scope. Three domains reduce narrow benchmark bias but do not cover all enterprise CI/CD and pricing behaviors.

Domain shift risk. Calibration may drift under different runner classes, cloud billing semantics, or workload mixes. Periodic recalibration remains necessary.

Graph branch maturity. Graph-only quality remains domain-sensitive, and adaptive routing often falls

back to `lstm_only` in real domains. Richer graph features are a clear next step.

Deployment envelope constraints. Results are demonstrated under a 4-GB class GPU and bounded memory controls. Different hardware may change runtime-quality tradeoffs.

Construct validity risk. High offline metrics alone do not guarantee governance usefulness. This work mitigates that risk by persisting policy reasons, actions, and decision provenance fields.

9 Reproducibility Notes

The reproducibility contract is explicit and executable:

- Fixed seed set: 42, 52, 62, 72, 82, 92, 102, 112, 122, 132.
- Fixed domain order: D0 Synthetic, L1 TravisTorrent, L2 BitBrains.
- Train-only scaler fitting and validation-only threshold locking.
- Seed-isolated run directories with per-run manifests and completion markers.
- Aggregate regeneration from immutable result artifacts.

Reference execution path:

```
bash run_ieee_trials.sh --ieee-10 \
  --laptop-safe --resume-epoch
```

Observed completion evidence in the completion summary CSV and the training terminal log confirms 10/10 completed runs from 2026-04-22 to 2026-04-24 (UTC).

10 Conclusion

CostGuard demonstrates that anomaly intelligence becomes materially more useful when integrated with policy-as-code and runtime governance. Under a fixed ten-seed, three-domain protocol, the system delivers stable high-quality anomaly detection, deterministic decision pathways, and deployable evidence traces on constrained hardware. The contribution is not a single model improvement; it is an integrated, reproducible control loop connecting ML scoring to enforceable CI/CD cost governance.

References

[1] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[2] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph Attention Networks," in *Proc. ICLR*, 2018.

[3] S. Mukherjee and S. Günnemann, "GhostLink: Latent Network Inference for Influence-Aware Recommendation," in *Proc. CIKM*, 2019.

[4] A. Zareian, S. Karaman, and S.-F. Chang, "Weakly Supervised Visual Semantic Parsing," in *Proc. CVPR*, 2020.

[5] R. Sridivya, M. S. Vijayabaskar, and K. Vimaladevi, "Machine Learning-Driven Improvements in Software Delivery Pipelines," *Journal of Theoretical and Applied Information Technology*, vol. 103, no. 19, 2025.

[6] N. Lokiny and P. Reddy, "Cost Optimization Strategies for DevOps Deployments in Cloud Environments Leveraging Machine Learning," *European Journal of Advances in Engineering and Technology*, vol. 8, no. 3, pp. 69–72, 2021.

[7] N. M. K. Koneru, "Optimizing CI/CD Pipelines for Multi-Cloud Environments: Strategies for AWS and Azure Integration," *Eastasouth Journal of Information System and Computer Science*, vol. 2, no. 3, pp. 288–310, 2025.

[8] J. R. M. Thason and S. K. Bahl, "Cloud Technology and DevOps: A Symbiotic Relationship," *Journal of Emerging Technologies and Innovative Research*, vol. 12, no. 3, 2025.

[9] L. R. Pereira, "The Impact of CI/CD in Business Value," M.Sc. thesis, Aalborg University, 2025.

[10] S. Deochake, "Cloud and AI Infrastructure Cost Optimization: A Comprehensive Review of Strategies and Case Studies," 2025.

[11] S. Deochake, "ABACUS: A FinOps Service for Cloud Cost Optimization," 2025.

[12] F. Klinaku, S. S. Stie, A. Hakamian, and S. Becker, "An Architectural View Type for Elasticity Policies—The Slingshot Approach," 2024.

[13] Z. Zhang and I. Matta, "SERFLOW: A Cross-Service Cost Optimization Framework for SLO-Aware Dynamic ML Inference," 2025.

[14] A. M. Kirubakaran *et al.*, "Governing Cloud Data Pipelines with Agentic AI," 2025.

[15] G. N. Junior and C. Marcon, "OPTIC: OPtimized Translation and Interaction for Cloud-Costs Using FOCUS Standardization and LLM Integration," *IEEE Access*, 2025.

[16] Open Policy Agent, "Open Policy Agent Documentation," 2024. [Online]. Available: https://www.openpolicyagent.org/docs/latest/

[17] FinOps Foundation, "FinOps Open Cost and Usage Specification (FOCUS)," 2025. [Online]. Available: https://focus.finops.org/

[18] T. S. Kamble, "CostGuard repository and experiment artifacts," 2026. [Online]. Available: https://github.com/tejasskamble/costguard-pade-finops

A Per-Seed Ensemble F1

Table 11: Per-seed ensemble F1 values used in aggregate analysis.

Seed	D0 Synthetic	L1 TravisTorrent	L2 BitBrains
42	0.8717	0.9059	0.9579
52	0.8972	0.9093	0.9608
62	0.8907	0.9080	0.9611
72	0.9004	0.9098	0.9601
82	0.9178	0.9090	0.9637
92	0.8935	0.9086	0.9596
102	0.8895	0.9096	0.9617
112	0.8931	0.9087	0.9610
122	0.8992	0.9096	0.9586
132	0.9091	0.9085	0.9614

B Extended Seed Metrics

Table 12: Extended per-seed ensemble metrics (F1 and PR-AUC).

Seed	D0 F1	D0 PR-AUC	L1 F1	L1 PR-AUC	L2 F1	L2 PR-AUC
42	0.8717	0.9552	0.9059	0.9491	0.9579	0.9717
52	0.8972	0.9668	0.9093	0.9435	0.9608	0.9776
62	0.8907	0.9645	0.9080	0.9366	0.9611	0.9770
72	0.9004	0.9645	0.9098	0.9436	0.9601	0.9770
82	0.9178	0.9781	0.9090	0.9434	0.9637	0.9794
92	0.8935	0.9667	0.9086	0.9417	0.9596	0.9763
102	0.8895	0.9643	0.9096	0.9439	0.9617	0.9770
112	0.8931	0.9644	0.9087	0.9351	0.9610	0.9767
122	0.8992	0.9563	0.9096	0.9393	0.9586	0.9731
132	0.9091	0.9669	0.9085	0.9433	0.9614	0.9771

C Representative Governance Scenarios

Table 13: Example policy-evaluation outcomes in CostGuard runtime.

Context	Condition	Expected action
Protected branch production deploy from PR	Context rule hit	`BLOCK`, independent of low CRS
Standard stage	$0.50 \leq CRS < 0.75$	`WARN` with owner notification
Sensitive stage with cost ceiling breach	Cost + context trigger	`AUTO_OPTIMISE` plus remediation guidance
Any stage	$CRS \geq 0.90$	`BLOCK` and pipeline halt

D Reproducibility Checklist

The following checklist was used to keep execution and reporting deterministic:

1. Freeze domain order to D0, L1, L2 for all seeds.
2. Keep seed set fixed at 42–132 and disallow ad hoc seed insertion.
3. Refit scalers only on train partitions for each seed/domain.
4. Tune thresholds only on validation outputs and freeze before test inference.
5. Persist per-seed manifests and stage completion markers.
6. Keep aggregate generation read-only over frozen per-seed artifacts.
7. Confirm 10/10 completion in `completion_summary.csv`.
8. Store orchestration output in `training_terminal_logs.md`.
9. Preserve hardware profile logs (`[GUARDIAN-PROFILE]` entries).
10. Preserve phase transitions and VRAM flush logs between model branches.

E Memory-Safe Design Rationale

CostGuard was executed under a constrained RTX 3050 profile. Three design choices were essential to completion reliability.

- **Adaptive batching:** batch sizes are reduced according to available VRAM and RAM tiers, with conservative limits for graph workloads.
- **Bounded preprocessing:** data loading avoids unsafe full materialization and prefers chunked processing where required.
- **Phase-level VRAM maintenance:** explicit memory flush checkpoints are run between LSTM and GAT phases to reduce carry-over pressure.

These controls are part of experiment design and reproducibility hygiene. They are reported because they materially influence whether multi-seed evidence can be completed and audited.